NWT Module 6 Exam Answers

Markup Languages and Server Side Programming

All 14 Module 6 questions from the supplied question bank, with exact question wording. Two-mark answers contain 4-5 sentence points or comparison rows; five-mark answers contain 5-8 detailed points.

2 Marks | Q17. Classify the different types of web pages (static and dynamic) and infer their suitability for various web applications.

Basis	Static web pages	Dynamic web pages
Content	A static page generally delivers the same stored content until its files are edited.	A dynamic page can change content according to user input, stored data, or application logic.
Generation	Its content is commonly written in HTML and CSS and served as a stored file.	Its content may be generated by server-side code or updated in the browser using JavaScript.
Interaction	It is suitable for presenting information that changes infrequently.	It supports features such as personalised dashboards, search, and shopping carts.
Processing	A basic static page does not require server-side processing to generate its content.	A dynamic application may require scripts, server processing, and database access.
Suitability	It suits simple portfolios, brochures, and information pages.	It suits e-commerce sites, banking portals, and social networking applications.

2 Marks | Q18. Examine the concept of Markup Language and analyze its significance in web technologies.

1. A markup language uses tags or annotations to describe the structure and meaning of a document's content.

2. HTML organises web content into elements such as headings, paragraphs, links, and tables.

3. XML uses custom element names to represent structured data for storage and exchange.

4. XHTML combines HTML vocabulary with the stricter syntax rules of XML.

5. Markup languages help browsers and other software interpret documents and separate structured content from its presentation.

2 Marks | Q19. Discuss the concept of URL rewriting with an example.

1. URL rewriting is a session-tracking technique in which a session identifier is included in a URL.

2. It allows a server to recognise requests belonging to the same session when cookies are unavailable.

3. In Servlets, response.encodeURL() can add a session identifier when URL-based tracking is needed.

4. For example, a URL may become /shop/cart;jsessionid=ABC123, where ABC123 identifies a server-side session.

5. Each relevant link must carry the identifier, and exposing session IDs in URLs can create security risks.

```
request.getSession();
String link = response.encodeURL("cart");
// Possible result when cookies are unavailable:
// cart;jsessionid=ABC123
```

2 Marks | Q20. Analyze the features of XHTML and infer its role in modern web development.

1. XHTML stands for Extensible HyperText Markup Language and expresses HTML using XML syntax.

2. Its element and attribute names must use the lowercase forms defined by the XHTML vocabulary.

3. All elements must be properly nested and closed, including empty elements such as <br />.

4. Attribute values must be quoted, and boolean attributes must include explicit values such as checked="checked".

5. Its consistent structure supports XML-based processing, although modern web development commonly uses HTML rather than XHTML.

2 Marks | Q21. Examine the applications of Java Servlets and analyze their role in generating dynamic web content.

1. Java Servlets are server-side Java components that process client requests and generate responses.

2. They read form parameters and can perform tasks such as validation, calculations, and authentication.

3. They interact with databases to retrieve or update information for applications such as shopping and student portals.

4. They generate dynamic responses in formats such as HTML, JSON, and XML according to the request.

5. They support session management to maintain user information across multiple requests.

2 Marks | Q22. Apply the standard structure of an HTML document to develop a basic web page.

1. An HTML document begins with a document-type declaration, such as <!DOCTYPE html>, to select standards-mode rendering.

2. The <html> element is the document's root element and contains its head and body sections.

3. The <head> section contains metadata, a page title, and references to styles or scripts.

4. The <body> section contains visible content such as headings, paragraphs, images, and links.

5. A basic page can be saved with a .html extension and opened in a web browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My College Page</title>
</head>
<body>
    <h1>Welcome to TCET</h1>
    <p>This is a basic HTML page.</p>
</body>
</html>
```

2 Marks | Q23. Compare XHTML with traditional HTML and justify the advantages of XHTML in web development.

Basis	Traditional HTML	XHTML
Syntax	HTML parsing is relatively forgiving and includes rules for handling malformed markup.	XHTML follows XML syntax and requires well-formed markup when processed as XML.
Case	HTML element names are case-insensitive in HTML syntax.	XHTML is case-sensitive and uses lowercase HTML vocabulary names.
Closing tags	Some HTML end tags may be omitted in defined situations.	XHTML requires elements to be closed, including empty elements.
Attributes	HTML allows certain unquoted values and shortened boolean attributes.	XHTML requires quoted values and explicit values for boolean attributes.
Advantages	Its forgiving syntax makes it convenient for general web authoring.	Its consistent syntax enables XML processing and encourages disciplined document structure.

Example: HTML permits <input disabled>, whereas XHTML uses <input disabled="disabled" />.

5 Marks | Q13. Analyze the syntax and semantics of XHTML, and evaluate its advantages and limitations in modern web development.

1. **Meaning:** XHTML is HTML expressed as XML, combining familiar web elements with strict document syntax. It was designed to make document structure consistent and suitable for XML-based processing.

2. **Document structure:** An XHTML 1.0 document uses an appropriate DOCTYPE and an html root with the XHTML namespace. Its head contains metadata and a title, while its body contains page content.

3. **Element rules:** Element names use lowercase, tags must close, and elements must be properly nested. For example, Text is properly nested, while crossed closing tags are invalid.

4. **Attribute rules:** Attribute values must be quoted, and boolean attributes cannot be shortened. For example, disabled="disabled" is used instead of a standalone disabled attribute.

5. **Semantics:** Semantics describes the meaning of elements, such as headings, paragraphs, and lists. Choosing a meaningful element helps software and readers understand the content rather than merely its visual appearance.

6. **Advantages:** Well-formed XHTML can be processed using XML tools, supporting consistent parsing and validation. Its explicit rules encourage organised and maintainable markup.

7. **Limitations:** Strict XML parsing can stop document processing when markup is malformed. Migrating permissive HTML requires care, and mixing HTML and XML processing assumptions can cause compatibility problems.

8. **Evaluation:** XHTML is useful where XML tooling or document-processing requirements are important. For general web development, HTML offers broader convenience, while semantic structure remains valuable in either format.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head><title>XHTML Example</title></head>
<body>
  <h1>Student Details</h1>
  <p><strong>Name:</strong> Aayush Gupta</p>
  <p>First line<br />Second line</p>
</body>
</html>
```

Illustration: The heading identifies a section, the paragraph holds text, strong marks importance, and the empty br element is explicitly closed.

5 Marks | Q14. Illustrate the types of HTML documents in detail with suitable examples.

1. **Meaning of document types:** In the XHTML context of your notes, the principal document types are Strict, Transitional, and Frameset. A DOCTYPE identifies the document definition against which the markup can be checked.

2. **Strict:** Strict permits structural markup and excludes deprecated presentational elements such as font. It encourages separation of document structure from visual styling through CSS.

3. **Strict example:** A paragraph styled using a CSS class is suitable for a Strict document. For example, Welcome keeps presentation in a stylesheet.

4. **Transitional:** Transitional includes structural elements and certain older presentational elements to support migration from legacy pages. It allows authors to modernise existing documents gradually instead of rewriting every element immediately.

5. **Transitional example:** Older markup such as Welcome belongs to the Transitional approach. This illustrates legacy compatibility rather than recommended modern authoring.

6. **Frameset:** Frameset supports documents that divide a browser window into separate frames. A frameset element replaces the usual body section and specifies the arrangement of frame documents.

7. **Frameset example:** A frameset with cols="25%,75%" loads a navigation document in one frame and a content document in another. Frames are obsolete in modern HTML and should be understood as a historical document type.

8. **Modern HTML:** Modern HTML uses the short <!DOCTYPE html> declaration rather than these three legacy DTD choices. This declaration selects standards mode and does not specify a Strict, Transitional, or Frameset DTD.

Type	Purpose	Typical illustration
Strict	It supports structured markup with presentation handled through CSS.	A paragraph uses a CSS class for colour.
Transitional	It supports migration of pages containing older presentational markup.	A legacy page uses the font element.
Frameset	It describes a window split into separate frame documents.	Navigation and content load in different frames.

```
<!-- Historical Frameset illustration -->
<frameset cols="25%,75%">
    <frame src="menu.html" />
    <frame src="content.html" />
</frameset>
```

5 Marks | Q15. Examine the concept of server-side programming and its significance in web development.

1. **Meaning:** Server-side programming executes application code on a web server to process client requests. The server sends the resulting HTML, JSON, or other response data back to the browser.

2. **Request processing:** The server reads request parameters, headers, and submitted form data. It then applies business rules, such as calculating a bill or checking a student's result.

3. **Database interaction:** Server-side code retrieves, inserts, and updates database records. A student portal can fetch a user's marks and generate a personalised result page.

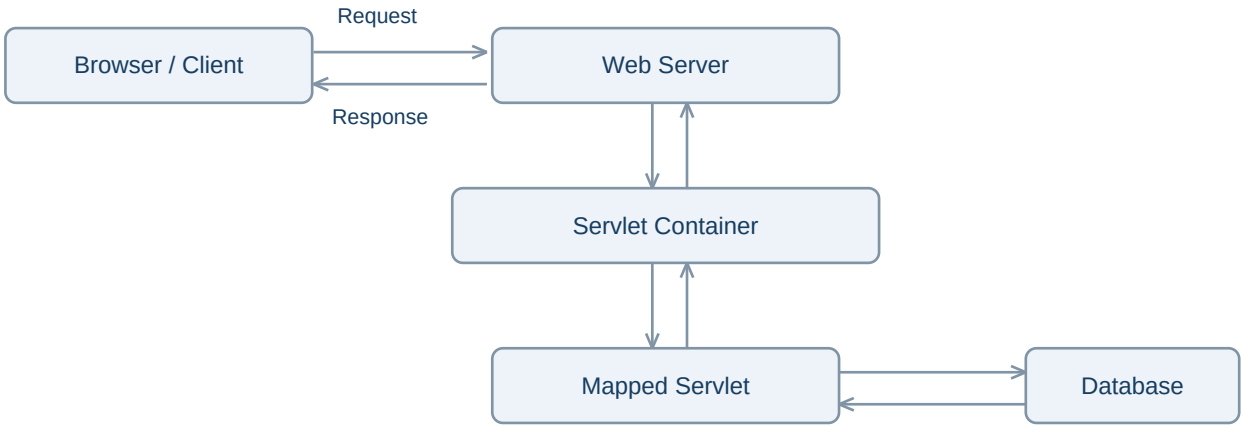
4. **Dynamic responses:** Response content can vary according to the user, request, and stored data. This supports search results, dashboards, shopping carts, and other data-driven features.

5. **Authentication and sessions:** Server-side code verifies user identity and stores relevant session information. Sessions preserve continuity across HTTP requests, such as remembering a logged-in user.

6. **Validation and controlled access:** The server checks submitted data and authorises access to protected resources. These checks are necessary because clients can alter requests or bypass browser-side validation.

7. **Technologies:** Java Servlets and PHP are examples of server-side technologies covered by this module's theme. A servlet container executes Java Servlets and manages their request handling and life cycle.

8. **Significance:** Server-side programming centralises business logic and enables controlled interaction with databases and services. Its limitations include server processing costs and the need to handle concurrency, errors, and resource usage carefully.



Example: A browser requests a student result; the servlet queries the database and returns a personalised HTML response.

5 Marks | Q16. Analyze the architecture of Java Servlets with a detailed explanation and diagram.

1. **Client:** A browser sends an HTTP request for a URL, such as /results. The request can carry parameters, form data, headers, and cookies.

2. **Web server:** The web server accepts the request and passes servlet-related processing to the servlet container. A server such as Tomcat can provide both HTTP handling and container functionality.

3. **Servlet container:** The container loads and initialises servlets, manages their life cycle, and routes requests. It also provides request and response objects and session support.

4. **URL mapping:** A deployment descriptor or a WebServlet annotation associates a URL pattern with a servlet. The container uses this mapping to identify the component that should handle the request.

5. **Request dispatch:** The container invokes service() for a request, and HttpServlet dispatches HTTP methods to handlers such as doGet() or doPost(). Each handler uses HttpServletRequest to inspect the incoming request.

6. **Application processing:** The servlet executes business logic and may access a database using JDBC. For example, it can retrieve a student's marks using a roll-number parameter.

7. **Response:** HttpServletResponse sets the status, content type, headers, and response body. The container and server return the response, which the browser displays or processes.

8. **Concurrency:** A servlet instance commonly handles multiple requests concurrently on separate threads. Request-specific data should therefore remain in local variables, while shared mutable data requires appropriate thread safety.

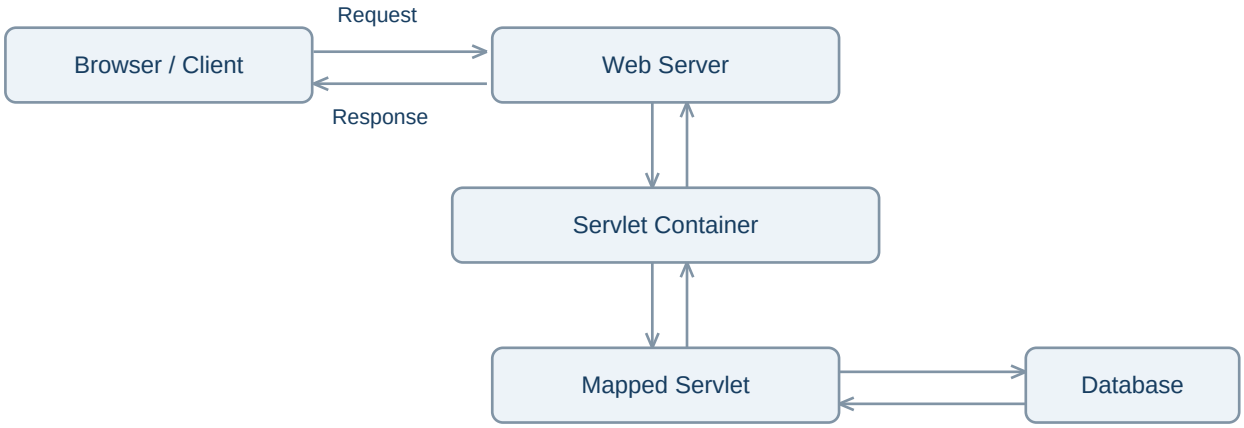


Diagram explanation: Requests move from browser to server to container to servlet, with optional database access. Responses return through the container and server to the browser.

5 Marks | Q17. Illustrate the process of generating dynamic content in Servlets with examples.

1. **Client request:** The browser sends a request to a URL mapped to a servlet. Parameters can identify what content is required, such as a product ID or a quantity.

2. **Invocation:** The container creates request and response objects and invokes the appropriate servlet handler. A GET request is normally processed through doGet().

3. **Reading input:** The servlet reads parameters using request.getParameter(). Input must be checked before it is used in calculations, output, or database operations.

4. **Processing:** The servlet performs calculations or retrieves data from a database or service. Different request values can therefore produce different responses from the same servlet.

5. **Response preparation:** The servlet sets the content type and obtains an output writer for text responses. HTML responses should specify the intended character encoding.

6. **Content generation:** Java code combines processed values with response markup. The servlet may generate HTML, JSON, XML, or another format required by the client.

7. **Delivery:** The container sends the completed response to the browser, which renders or processes it. Examples include personalised dashboards, search results, and dynamically calculated bills.

```
import java.io.IOException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/bill")
public class BillServlet extends HttpServlet {
  protected void doGet(HttpServletRequest request,
      HttpServletResponse response) throws IOException {
    response.setContentType("text/html;charset=UTF-8");
    int quantity;
    try {
      quantity = Integer.parseInt(request.getParameter("qty"));
      if (quantity < 1) throw new NumberFormatException();
    } catch (NumberFormatException e) {
      response.sendError(400, "Enter a positive quantity.");
      return;
    }
    long total = quantity * 50L;
    response.getWriter().println("<h2>Total: Rs. " + total + "</h2>");
  }
}
```

Example: A request to /bill?qty=3 produces "Total: Rs. 150", while qty=5 produces "Total: Rs. 250". The price is fixed in this illustration, and the total changes with the validated quantity.

Examples use the javax.servlet package notation found in the course material; a Jakarta-based environment uses corresponding jakarta.servlet imports.

5 Marks | Q18. Compare the types of cookies in Servlets and examine their role in session management.

1. **Meaning:** A cookie is a small name-value record stored by a browser for a website. The server sends it using a Set-Cookie response header, and the browser returns it with applicable later requests.

2. **Session cookies:** A session cookie has no persistent expiry and is intended to last for the browser session. It commonly carries a session identifier, although browser session-restoration behaviour can affect how long it remains available.

3. **Persistent cookies:** A persistent cookie has a specified lifetime and can remain across browser restarts until expiry or deletion. It is useful for remembering settings such as a preferred language or theme.

4. **Servlet creation:** A Servlet creates a Cookie object and adds it to the response using response.addCookie(). A positive setMaxAge() specifies a lifetime in seconds, while a negative value makes it non-persistent.

5. **Reading and deleting:** request.getCookies() retrieves cookies sent with a request and may return null when none are present. To delete a cookie, send it with a maximum age of zero and matching path and domain settings.

6. **Session management:** A container commonly uses a JSESSIONID cookie to associate the browser with a server-side HttpSession. The cookie stores an identifier, while user-specific session attributes stay on the server.

7. **Protection and limitations:** HttpOnly restricts JavaScript access, and Secure limits transmission to secure connections. Cookies may be blocked or deleted, so session-tracking behaviour must account for their availability.

Basis	Session cookie	Persistent cookie
Lifetime	It is intended to last for the browser session.	It remains until its specified expiry or deletion.
Servlet setting	A negative maximum age makes it non-persistent.	A positive maximum age sets its lifetime in seconds.
Use	It commonly supports temporary session identification.	It commonly remembers preferences across visits.

```
Cookie preference = new Cookie("theme", "light");
preference.setMaxAge(7 * 24 * 60 * 60); // Seven days
preference.setPath("/");
response.addCookie(preference);

// Container-managed session data:
request.getSession().setAttribute("username", "Aayush");
```

Cookie exchange: The server sends a session-ID cookie; the browser returns that cookie with later requests; the container uses the ID to locate the server-side session.

5 Marks | Q19. Examine the life cycle of a Servlet with a detailed diagram and suitable example.

1. **Container control:** The servlet life cycle describes its creation, initialisation, request handling, and removal. The servlet container manages these stages rather than requiring the application to call them directly.

2. **Class loading:** The container loads the servlet class at application startup or when it is first requested, depending on configuration. Loading makes the servlet's code available for execution.

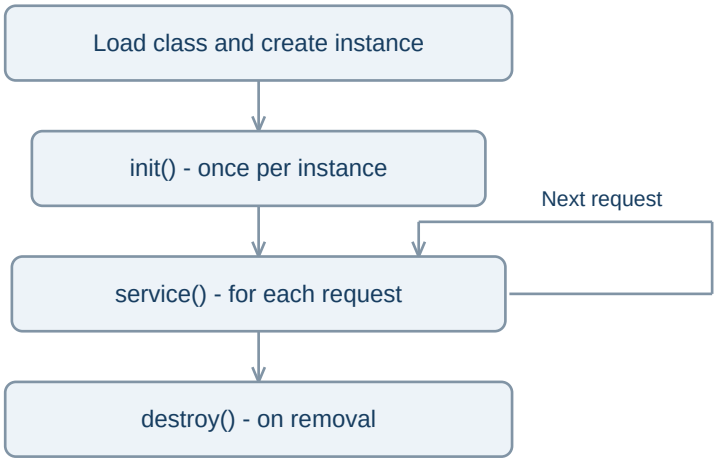
3. **Instantiation:** The container creates an instance of the servlet class. The same instance commonly handles many requests during its lifetime.

4. **Initialisation:** The container calls init() once for that servlet instance before it handles requests. This method performs one-time setup, such as reading configuration or preparing resources.

5. **Request handling:** The container calls service() for each incoming request. For HttpServlet, service() dispatches to handlers such as doGet() and doPost() according to the HTTP method.

6. **Repeated requests:** After processing a request, the servlet remains available to process further requests. Multiple request threads may use the same instance, so shared mutable fields require care.

7. **Destruction:** The container calls destroy() when it removes the servlet from service. This method releases resources, and the instance can later become eligible for garbage collection.




```
import java.io.IOException;
import javax.servlet.http.*;

public class LifeCycleServlet extends HttpServlet {
  public void init() {
    getServletContext().log("Initialised once");
  }
  protected void doGet(HttpServletRequest request,
      HttpServletResponse response) throws IOException {
    response.setContentType("text/plain;charset=UTF-8");
    response.getWriter().println("Request handled");
  }
  public void destroy() {
    getServletContext().log("Servlet removed");
  }
}
```

Example: init() runs once for the instance; repeated GET requests run doGet() through service(); destroy() runs when the servlet is removed.

References: Supplied NWT question bank, Unit 6 notes, and NWT chapter 6 notes.